INSTITUTE OF ACCOUNTANCY ARUSHA

INDIVIDUAL ASSIGNMENT

STUDENT NAME : GOODLUCK V. VINCENT

REG NO :BCS-01-0131-2023

PROGRAM : BCS III

MODULE NAME : OPEN SOURCE SOFTWARE

CODE : ITU 08102

FACILITATOR : CHIBELENJE (MR.STANLEY)

SEMESTER : ONE

ACCADEMIC YEAR : 2025/2026

Task 1: Four Compatibility and Integration Challenges

1. Software Version and Operating System Incompatibility The open-source LMS may be built to run on specific versions of PHP, Python, or Java, while the existing institutional servers run older or incompatible operating system versions. For example, Moodle may require PHP 8.x while the server runs PHP 7.x, causing functional failures. Similarly, the SIS may have been developed for a Windows environment while the LMS runs on Linux, creating environment-level conflicts that require significant reconfiguration before the systems can coexist.

2. Data Format and Database Structure Differences The LMS, SIS, and Library Management System likely use different database engines (e.g., MySQL vs. Oracle vs. PostgreSQL) and store data in different formats and schemas. Student records in the SIS may use a specific ID format or date structure that does not map directly to the LMS database fields. This mismatch causes data corruption, duplication, or loss during synchronization, making it difficult to maintain consistent academic records across systems.

3. Limited or Outdated APIs Some legacy institutional systems such as older library management platforms or student information systems were built before modern API standards (REST, SOAP) became common. Their APIs may be poorly documented, use deprecated protocols, or lack endpoints needed for integration. For instance, the library system may not expose an API for checking out materials linked to a student's LMS account, forcing the team to build custom workarounds that are fragile and difficult to maintain over time.

4. Authentication and Single Sign-On (SSO) Incompatibility The institutional authentication system may use an older protocol such as LDAP or a proprietary identity provider, while the open-source LMS supports SAML 2.0 or OAuth 2.0. This incompatibility prevents seamless staff and student logins across all systems. Users are forced to maintain separate credentials for each system, reducing usability and creating security vulnerabilities. Implementing a unified SSO solution requires additional configuration and may not be straightforwardly supported by all legacy systems involved.

Task 2: Three Practical Technical Solutions for Compatibility and Integration

1. Implementation of a Middleware Integration Layer (API Gateway) A middleware layer or API gateway acts as a translator and coordinator between the LMS, SIS, and library system. Tools such as Apache Camel, MuleSoft (community edition), or custom RESTful middleware can receive data from one system, transform it into the format expected by another, and forward it accordingly. For example, when a student registers in the SIS, the middleware can automatically create a corresponding user account in the LMS using mapped data fields. This decouples the systems so that changes in one do not directly break another, significantly improving maintainability and reducing integration complexity.

2. Adoption of Open Data Standards and Common Data Formats The ICT team should standardize data exchange across all systems using widely accepted open formats such as JSON, XML, or IMS Global standards like Learning Tools Interoperability (LTI) and IMS Common Cartridge. LTI, for instance, allows external tools and content from the library or SIS to be embedded directly into the LMS without requiring full system integration. By agreeing on a common data schema for student records, the team eliminates mapping errors and ensures that data flowing between the SIS, LMS, and library system is consistent and interpretable by all parties without custom transformation logic for every integration point.

3. Deployment of a Centralized Identity and Access Management (IAM) System To resolve authentication incompatibilities, the institution should deploy an open-source IAM solution such as Keycloak or Shibboleth, which supports multiple protocols including LDAP, SAML 2.0, and OAuth 2.0. This acts as a central identity provider that all systems authenticate against. The LMS, SIS, and library system each connect to Keycloak rather than directly to each other, enabling true Single Sign-On (SSO). Staff and students log in once and gain seamless access to all integrated systems. Keycloak can bridge legacy LDAP directories with modern OAuth-based applications, effectively solving cross-system authentication incompatibility without replacing the existing directory infrastructure.

Task 3: Analysis of Unmaintained Open-Source Plugins

Two Benefits

1. Immediate Cost and Time Savings Unmaintained plugins often provide ready-built functionality that would otherwise require significant development effort and resources to build from scratch. For an institution with a limited ICT budget, using an existing plugin even one no longer actively updated to handle a specific LMS feature such as a custom grade display or library catalog widget can accelerate deployment and reduce initial implementation costs. The plugin code is typically open and available, so the institution can use it freely without licensing fees.

2. Access to Functional Legacy Features Some unmaintained plugins were built for very specific institutional needs and still perform their intended function well within the current system version. If the LMS version being adopted is older or stable, an unmaintained plugin designed for that version may continue to work reliably. Institutions can benefit from community tested functionality without waiting for a maintained alternative to be developed, particularly for niche integrations that commercial vendors do not support.

Two Risks

1. Security Vulnerabilities Since the plugin is no longer actively maintained, discovered security flaws will not be patched by the original developers. Malicious actors who identify vulnerabilities in widely used unmaintained plugins can exploit them to gain unauthorized access to student records, manipulate grades, or compromise the entire LMS server. In an institutional environment handling sensitive academic and personal data, this poses a serious legal and reputational risk, particularly in the context of national data protection regulations.

2. Incompatibility with Future System Updates As the LMS core continues to be updated, unmaintained plugins will eventually break because they are not updated to conform to new APIs, database structures, or security requirements of newer LMS versions. This creates a long-term sustainability problem: the institution either remains on an outdated LMS version to preserve plugin functionality exposing itself to broader security risks or upgrades the LMS and

loses the plugin's functionality entirely. Both outcomes create operational disruption and additional unplanned maintenance costs.

Task 4: How Open Standards, Documentation, and Version Control Support Interoperability and Sustainability

Open Standards Open standards such as LTI (Learning Tools Interoperability), REST APIs, OAuth 2.0, and IMS Global specifications provide universally agreed-upon rules for how software systems communicate and exchange data. When the LMS and its integrated systems are built on open standards, any compliant tool or system can connect to them without requiring proprietary adapters or vendor-specific configurations. This directly enables interoperability the LMS can connect to a new library system or SIS regardless of who built it, as long as both conform to the same standard. Open standards also promote long term sustainability because they are governed by independent bodies and not subject to a single vendor's commercial decisions, meaning the institution's investment in integration work retains its value even as individual system components change over time.

Proper Documentation Thorough documentation of system architecture, API endpoints, data schemas, configuration settings, and integration logic is essential for both interoperability and sustainability. When systems are well documented, new developers or ICT staff can understand how components interact and safely extend or modify them without breaking existing integrations. Documentation also facilitates onboarding of external developers or community contributors who may help maintain or improve the open source LMS. In the context of Karume University, clear API documentation for the SIS and library system would allow the LMS integration team to build reliable connectors without depending on tribal knowledge held by individual staff members. Poor documentation, by contrast, causes integration failures whenever key personnel leave and creates duplication of effort as developers rediscover solutions independently.

Version Control Version control systems such as Git enable teams to track every change made to the LMS codebase, custom plugins, configuration files, and integration scripts over time. This supports sustainability by ensuring that no change is irreversible if a new plugin update breaks an integration, the team can roll back to a previous working state quickly. Version control also supports collaboration, allowing multiple developers to work on different parts of the system

simultaneously without overwriting each other's contributions, through branching and merging strategies. For open-source development specifically, platforms like GitHub or GitLab enable the institution's customizations to be shared with the broader open-source community, attracting external contributions that reduce the maintenance burden on internal staff. Combined with proper tagging of releases and changelog documentation, version control gives institutional stakeholders a clear audit trail of what changed, when, and